

Real-world (act out a canary release) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `canary_release_practice.py`: a canary sends a small share of real traffic to the new version and compares it against the old one, before everyone gets it. Acting it out on your own project makes it something you can explain concretely. **If not:** "we'd do a canary release" would stay a phrase you can't back up with numbers.

All examples below use this shared setup — 20 fake test requests, and a `run_eval_suite` stub where `"v2"`'s prompt scores better than `"v1"`'s:

```
# canary_release_practice.py
import random

prompts = {
    "v1": "Answer the customer's question.",
    "v2": "Answer the customer's question politely and cite the source.",
}
test_requests = list(range(20)) # 20 stand-in requests

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}
```

Each example below calls `random.seed(...)` with a fixed number so the "random" routing is the same, reproducible split every time you run it — remove the seed line for genuinely random routing in real use.

Basic Version

Approach 1 — the direct way

Story: route about 20% of requests to v2, score both groups, compare averages — the whole idea in one loop. **If not:** you'd have no way to see v2 on real traffic before everyone got it.

```
# canary_release_practice.py
import logging
```

```

import os

logging.basicConfig(level=logging.INFO)

CANARY_SHARE = float(os.getenv("CANARY_SHARE", "0.2"))
random.seed(1)

def route_and_score(test_requests, prompts):
    results = []
    canary_count = 0
    for request in test_requests:
        if random.random() < CANARY_SHARE:
            version = "v2"
            canary_count += 1
        else:
            version = "v1"
        score = run_eval_suite(prompts[version])["score"]
        results.append({"version": version, "score": score})
    logging.info(
        f"Routed {len(results)} requests, {canary_count} of them to canary v2"
    )
    return results

def average_by_version(results, version):
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r["score"])
    return sum(matching) / len(matching)

results = route_and_score(test_requests, prompts)
print("v1 avg:", average_by_version(results, "v1"))
print("v2 avg:", average_by_version(results, "v2"))

```

Expected output (with no `CANARY_SHARE` set, so the default `0.2` is used):

```

INFO:root:Routed 20 requests, 5 of them to canary v2
v1 avg: 0.5999999999999999
v2 avg: 1.0

```

(`0.5999999999999999` is `0.6` with the tiny rounding error binary floats have — adding fifteen `0.6`s and dividing doesn't land exactly on `0.6`.) This version works correctly for what the exercise asks. It's missing type hints, doesn't track cost, and would crash with a `ZeroDivisionError` if a run happened to route zero requests to `"v2"` — try `random.seed(0)` instead of

`random.seed(1)` : that 20-request run sends nothing to `v2` , the log line says `0 of them to canary v2` , and the next line crashes. All fine for a first working version, all fixed below.

Revision from Doc01 (Basic): two basic habits come back. The canary share is read with `os.getenv("CANARY_SHARE", "0.2")` — a real rollout moves from 5% to 20% to 50%, and each step should be a settings change, not a code edit. And the routing result is logged with `logging.info(...)` at `INFO` , because splitting traffic is normal progress. That one line is also the first thing to check when an average looks strange: if it says `0 of them to canary v2` , you know why before you even read the crash.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a `canary_share` parameter, cost tracking, and a `summarize()` function

Story: quality isn't the only number that matters — a better answer that costs twice as much may not be worth it. And an empty canary group should say *why* it's empty. **If not:** you'd promote on score alone, and a small run would crash with a bare `ZeroDivisionError`.

```
# canary_release_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)
console = logging.StreamHandler()
console.setLevel(logging.INFO)
logfile = logging.FileHandler("canary.log")
logfile.setLevel(logging.DEBUG)
logger.addHandler(console)
logger.addHandler(logfile)

class EmptyGroupError(Exception):
    pass

random.seed(1)

def route_and_score(
    test_requests: list, prompts: dict, canary_share: float = 0.2
) -> list[dict]:
    results = []
    for request in test_requests:
        # how: random.random() is between 0 and 1, so this is true
        # for about canary_share of all requests
        if random.random() < canary_share:
            version = "v2"
        else:
            version = "v1"
        logger.debug(f"request {request} -> {version}")
        eval_result = run_eval_suite(prompts[version])
        results.append({
            "version": version,
            "score": eval_result["score"],
```

```

        "cost": len(prompts[version]) * 0.0001,
    })
    logger.info(
        f"Routed {len(results)} requests with canary_share={canary_share}"
    )
    return results

def summarize(results: list[dict], version: str) -> dict:
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r)
    if not matching:
        # why: an empty group is a rollout problem, not a math bug –
        # say so, instead of crashing on division by zero
        raise EmptyGroupError(
            f"No requests were routed to {version} – "
            "send more requests or raise canary_share"
        )

    scores = []
    costs = []
    for r in matching:
        scores.append(r["score"])
        costs.append(r["cost"])

    return {
        "version": version,
        "count": len(matching),
        "avg_score": sum(scores) / len(scores),
        "avg_cost": sum(costs) / len(costs),
    }

results = route_and_score(test_requests, prompts)
control = summarize(results, "v1")
canary = summarize(results, "v2")
print(control)
print(canary)

```

Expected output:

```

Routed 20 requests with canary_share=0.2
{'version': 'v1', 'count': 15, 'avg_score': 0.6, 'avg_cost': 0.0031}
{'version': 'v2', 'count': 5, 'avg_score': 1.0, 'avg_cost': 0.006}

```

(the real printed float carries extra trailing digits from binary floating point, like `0.0030999999999999995` — rounded here for readability)

The terminal shows only the `INFO` line; `canary.log` also holds one `DEBUG` line per request, like `request 0 -> v2` and `request 1 -> v1`. With `random.seed(0)`, `summarize(results, "v2")` now stops with a clear `EmptyGroupError` ("No requests were routed to v2 — send more requests or raise `canary_share`") instead of a bare `ZeroDivisionError`.

Revision from Doc01 (Intermediate): two habits come back. First, Doc01's "logger with two output levels" exercise: the terminal gets one `INFO` summary, while `canary.log` keeps a `DEBUG` line for every single routing decision. When a customer says "I got the new answer style yesterday", the file tells you which version served each request — 20 lines in the terminal would just be noise. Second, a named error with a clear message: an empty group is not a bug in the math, it's a problem with the *rollout* (too few requests or too small a share), and `EmptyGroupError` says exactly that, where `ZeroDivisionError: division by zero` says nothing useful.

Difference from Basic: Approach 1 takes `canary_share` as a function parameter instead of a module-level setting, logs every routing decision to a file, raises a named `EmptyGroupError` instead of crashing on an empty group, tracks a per-request cost estimate alongside score, and replaces `average_by_version` with `summarize()`, which returns a full dict (`count`, `avg_score`, `avg_cost`) instead of just one number — `count` in particular matters: an average from 2 requests means much less than one from 200.

Which one should you actually write? Approach 1 of the Intermediate version — per-group counts, scores and costs, with a named error for an empty group — is the minimum for a real rollout. Before promoting, look at `count` too: if the canary group is only a handful of requests, run more before you trust its average. In your real Project 5 rollout, swap the stand-in cost (`len(prompt) * 0.0001`) for real per-request token cost, and the fake scores for your Doc13 suite.